# Bounding the Quadtree Size

*Why 2m and 2n are upper bounds on the padded dimensions — not padded dimensions themselves*

> **KEY POINT**
>
> 2*m* and 2*n* do **not** need to be powers of two. They are simply upper bounds on the powers of two used for padding.

## 1. Proof that M′ ≤ 2*m*

Let

$$M' = 2^{\lceil \log_2 m \rceil},$$

where $m \geq 1$ is an integer.

By the definition of the ceiling function,

$$\lceil \log_2 m \rceil < \log_2 m + 1.$$

Therefore, exponentiating both sides with base 2 (a strictly increasing function),

$$2^{\lceil \log_2 m \rceil} < 2^{\log_2 m + 1}.$$

Using $2^{\log_2 m} = m$,

$$M' = 2^{\lceil \log_2 m \rceil} < 2m.$$

Hence, in fact,

$$M' < 2m$$

for every positive integer $m$. Therefore the weaker statement

$$M' \leq 2m$$

is certainly true.

The same argument gives

$$N' < 2n.$$

## 2. Why This Makes Intuitive Sense

Suppose $2^k$ is the smallest power of two that is at least m. Then

$$2^{k-1} < m \leq 2^k.$$

Multiplying the left inequality by 2,

$$2^k < 2m.$$

$'$ $2^k$,
Since M =

$$M' < 2m.$$

**Examples**

| *m* | M′ | 2*m* | Inequality |
|---|---|---|---|
| 4 | 4 | 8 | $4 < 8$ |
| 5 | 8 | 10 | $8 < 10$ |
| 10 | 16 | 20 | $16 < 20$ |
| 17 | 32 | 34 | $32 < 34$ |

So 2*m* is not itself a padded dimension. It is merely a convenient upper bound.

## 3. Applying This to the Quadtree

The padded dimensions satisfy

$$M' < 2m, \quad N' < 2n.$$

Therefore the number of padded cells (and hence the maximum possible number of leaf cells, if every cell becomes a leaf) is

$$L = M' N'.$$

Thus

$$L < (2m)(2n) = 4mn.$$

So

$$L < 4mn.$$

> **ONE IMPORTANT CORRECTION TO THE NEXT STEP**
>
> Your subsequent argument
>
> $$T = 1 + 4 + 4^2 + \ldots + 4^H = (4^{H+1} - 1)/3 \approx (4/3)L$$
>
> is correct **only** when the quadtree is a complete/full 4-ary tree with all leaves at the same depth.

If you only need a non-strict bound, you can write

$$L \leq 4mn.$$

## 4. A Cleaner Bound via Full 4-ary Trees

There is an even cleaner way to obtain the bound. For a full 4-ary tree with $L$ leaves, let $I$ be the number of internal nodes. Since every internal node has exactly four children,

$$4I = I + L - 1,$$

because a tree with $I + L$ total nodes has $I + L - 1$ edges. Hence

$$3I = L - 1,$$

so

$$I = (L - 1)/3.$$

Therefore the total number of nodes is

$$T = I + L = (L - 1)/3 + L = (4L - 1)/3.$$

Since $L < 4mn$,

$$T = (4L - 1)/3 < (16mn - 1)/3 < (16/3)mn.$$

Thus a convenient asymptotic bound is

$$T = O(mn).$$

## 5. Summary of the Overall Reasoning

$$M' < 2m, \quad N' < 2n \longrightarrow L = M'N' < 4mn \longrightarrow T = (4L - 1)/3 < (16/3)\ mn$$

> **THE CRUCIAL IDEA**
>
> 2*m* and 2*n* are **bounds**, not powers of two themselves.

---



## 304. Range Sum Query 2D - Immutable

Solved

Medium Topics Companies

Given a 2D matrix `matrix`, handle multiple queries of the following type:

- Calculate the **sum** of the elements of `matrix` inside the rectangle defined by its **upper left corner** `(row1, col1)` and **lower right corner** `(row2, col2)`.

Implement the `NumMatrix` class:

- `NumMatrix(int[][] matrix)` Initializes the object with the integer matrix `matrix`.
- `int sumRegion(int row1, int col1, int row2, int col2)` Returns the **sum** of the elements of `matrix` inside the rectangle defined by its **upper left corner** `(row1, col1)` and **lower right corner** `(row2, col2)`.

You must design an algorithm where `sumRegion` works on `O(1)` time complexity.

```
/*
splitting convention followed

        |   |   midCol
        |   |     |
    m1  |   |     |   m2
        |   |     |
    ----+---+-----+-----
        |   |     |
    m4  |   |     |   m3
        |   |     |
        |   |   midRow

*/
```

```cpp
class SegmentTree {
    public:
    int m;
    int n;
    int size;
    vector<int> tree;
    vector<vector<int>> sourceArray;

    // pass only unique parameters , the parameters with f(some other parameter) calculate inside.
    SegmentTree (int m, int n, vector<vector<int>> &sourceArray) {
        this->m = m;
        this->n = n;
        this->size = 4 * m * n + 1;
        this->tree = vector<int>(4 * m * n + 1, 0);
        this->sourceArray = sourceArray;
    }

    // since we already calculated the max of index will be 4*n (with dummy nodes),
    // we dont need to add any check for out of bounds of index. (not sure check while debugging)
    int build(int index, pair<int,int> low, pair<int,int> top) {
        // Invalid rectangle , prevents out of bounds
        if (low.first > top.first || low.second > top.second)
            return 0;

        // single element base
        if (low.first == top.first && low.second == top.second) { // not sure (debug if needed)

            return tree[index] = sourceArray[low.first][low.second];
        }

        // double element base
        // (Handled cleanly via recursive subdivision)

        int midRow = low.first + (top.first - low.first) / 2;
        int midCol = low.second + (top.second - low.second) / 2;

        // child indices from 4 * index to 4 * index + 1, 4 * index + 2, 4 * index + 3, and
        // 4 * index + 4 (using 4 * index as a child caused a recursive collision where node 0 was its own child).
        int m1 = build(4 * index + 1, low, {midRow, midCol});
        int m2 = build(4 * index + 2, {low.first, midCol + 1}, {midRow, top.second});
        int m3 = build(4 * index + 3, {midRow + 1, midCol + 1}, top);
        int m4 = build(4 * index + 4, {midRow + 1, low.second}, {top.first, midCol});

        return tree[index] = m1 + m2 + m3 + m4;
    }

    void build () {
        cout << "Sum of elements : " << build(0, {0,0}, {m - 1, n - 1}) << endl;
    }

    bool IsInvalidQuery(pair<int,int> qlow, pair<int,int> qtop , pair<int,int> low, pair<int,int> top) {
        // bounds exceed or [low, top] bounds invalid

        return ( qlow.first < 0 || qlow.second < 0 || qlow.first >= m || qlow.second >= n ) ||
               ( qtop.first < 0 || qtop.second < 0 || qtop.first >= m || qtop.second >= n ) ||
               ( qlow.first > qtop.first || qlow.second > qtop.second );
    }

    // -1 indicates something went wrong
    int query (int index, pair<int,int> qlow, pair<int,int> qtop, pair<int,int> low, pair<int,int> top) {
        // bounds exceed or [low, top] bounds invalid
        if(IsInvalidQuery(qlow, qtop, low, top)) {
            cout << "ERROR !" << endl;
            return 0;
        }

        // Check if current node's rectangle is completely outside the query range
        if (top.first < qlow.first || low.first > qtop.first ||
            top.second < qlow.second || low.second > qtop.second) {
            return 0;
        }
```

```
        // total overlapp
        if((low.first >= qlow.first && top.first <= qtop.first ) &&
            (low.second >= qlow.second && top.second <= qtop.second)) {

            return tree[index];
        }

        int midRow = low.first + (top.first - low.first) / 2;
        int midCol = low.second + (top.second - low.second) / 2;

        int sq1 = query(
            4 * index + 1,
            qlow,
            qtop,
            low,
            {midRow, midCol}
        );

        int sq2 = query(
            4 * index + 2,
            qlow,
            qtop,
            {low.first, midCol + 1},
            {midRow, top.second}
        );

        int sq3 = query(
            4 * index + 3,
            qlow,
            qtop,
            {midRow + 1, midCol + 1},
            top
        );
```

```
        int sq4 = query(
            4 * index + 4,
            qlow,
            qtop,
            {midRow + 1, low.second},
            {top.first, midCol}
        );

        return sq1 + sq2 + sq3 + sq4;

    }

    int query (pair<int,int> qlow, pair<int,int> qtop) {
        return query(0, qlow, qtop, {0,0}, {m - 1, n - 1});
    }
};

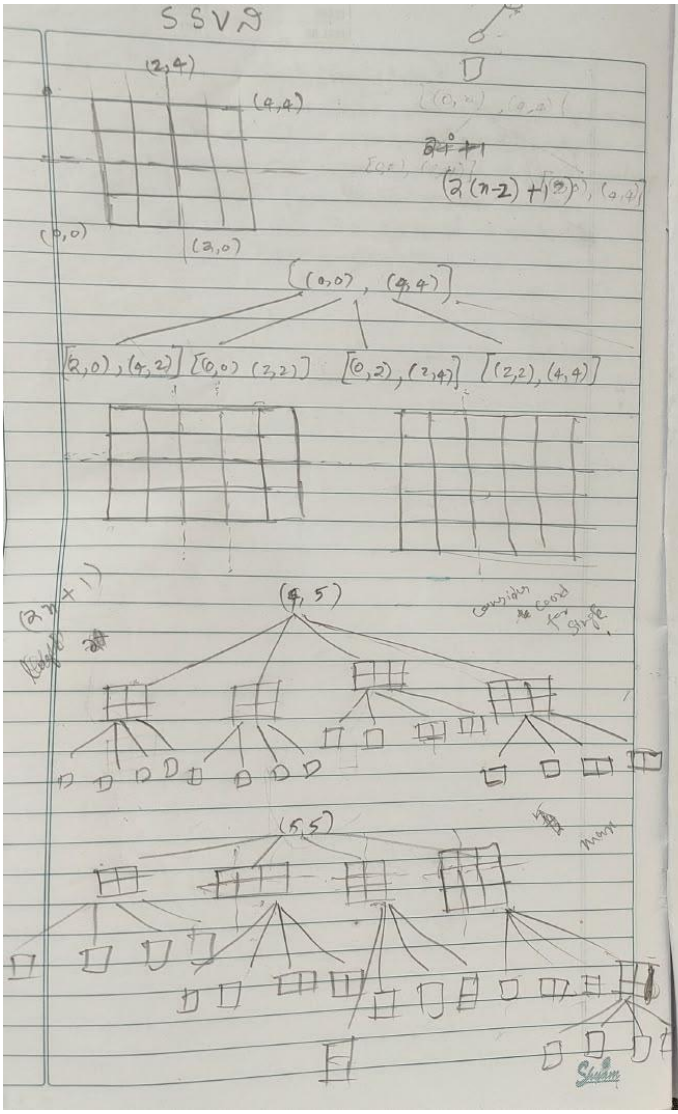
class NumMatrix {
public:
    SegmentTree treeInstance;
    vector<vector<int>> matrix;

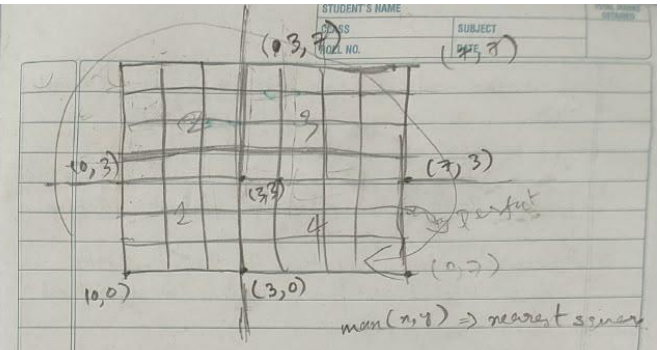
   //you can avoid the initializer-list syntax, but then you must make treeInstance a pointer because SegmentTree has no default constructor.
    NumMatrix(vector<vector<int>>& matrix)
        : matrix(matrix),
          treeInstance(matrix.size(), matrix[0].size(), matrix) {

        cout << "Building" << endl;
        treeInstance.build();
        cout << "Done building" << endl;
    }

    int sumRegion(int row1, int col1, int row2, int col2) {

        cout << "Start query" << endl;
        return treeInstance.query({row1,col1} , {row2,col2});
    }
};
```




max(x,y) ⇒ nearest square

* $4(m \times n) - 1$
⇒ $4(4n) - 1$
⇒ $16n$

Searches [l, r],
query [2l, 2r]

⇒ for other half use top, lowers use low

⇒ Segment trees use extra space to accomodate the full binary tree nature, but the number of elements may not be of 'n' with perfect square

⇒ Use [l, r] to trim & check with [2l, 2r].
[l, r] ⇒ space of Seg tree, space of array given by user
1D ⇒ (0, 4n-1)
2D ⇒ (0, 16n-1)
query [2l, 2r]

{x-first, y-second} ⇒ x can be equal to y.

better to define a class like class { becomes int x; int y; };

⇒ In 2D consider, three base case

Shivam